

0.29.2. Revenir au départ avec va_a — doublé, deux positions

L'aller-retour du A (en haut), puis celui du B (en bas).

1. Ce qu'il faut comprendre

C'est le 0.29 et le 0.29.1 réunis : les deux allers-retours.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1739 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 0;      // la colonne du A
```

2. UNE VARIABLE — Y

```
uint8_t y = 0;      // sa ligne
```

3. UNE VARIABLE — XB

```
uint8_t xb = 0;     // la colonne du B
```

4. UNE VARIABLE — YB

```
uint8_t yb = 10;    // sa ligne
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    va_a(x, y, ALPHABET[0], 10, 5);
    va_a(x, y, ALPHABET[0], 0, 0);
    va_a(xb, yb, ALPHABET[1], 10, 15);
    va_a(xb, yb, ALPHABET[1], 0, 10);

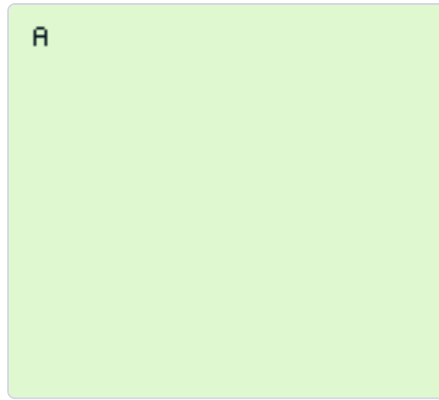
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

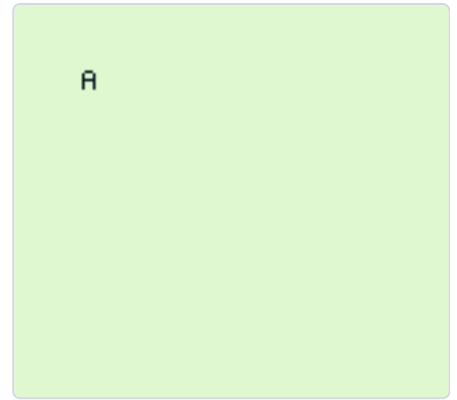
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



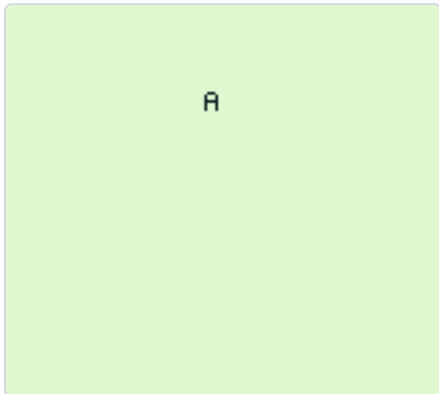
la toute première image — la console vient de démarrer — image 1



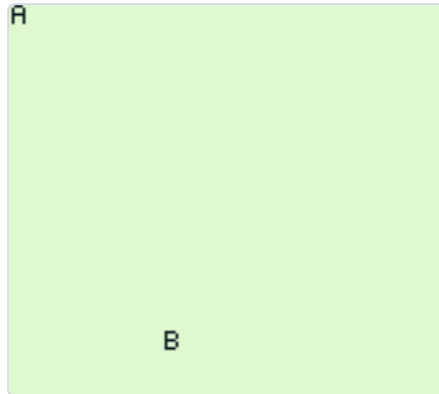
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le A va et revient en haut, puis le B en bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A finit en (0, 0) (0,0)
- ✓ le B passe par (10, 15)
- ✓ le B finit en (0, 10) (0,10)

Ensuite : **0.30. Un pas sans attendre : un_pas** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++